

Taskify

Backend API Development Roadmap

Day-wise implementation plan for Node.js + Express.js + MongoDB/Mongoose + Postman

Purpose

This document converts the existing Taskify API Specification, FRD, HLD, LLD and Database Design into a practical backend implementation sequence. The focus is only on backend API development and Postman testing.

Architecture

Request → Route → Authentication → Authorization → Validation → Controller → Service → Model → MongoDB → Response

Primary Roles: SUPER_ADMIN, ADMIN, EMPLOYEE

Base URL: /api

1. Backend Development Strategy

The backend should be developed incrementally. Finish one domain, test it in Postman, and then move to the next domain. Do not build the frontend at the same time for this roadmap.

Core backend architecture

```
HTTP Request
↓
Route
↓
Authentication Middleware
↓
Authorization Middleware
↓
Validation
↓
Controller
↓
Service
↓
Mongoose Model / Data Access
↓
MongoDB
↓
Response
```

Important implementation rules

- Controllers should receive request data, call services, and return HTTP responses; large business workflows belong in services.
- Authorization uses Role + Data Scope, not role alone.
- Important task-state changes generate Task History.
- Historical records should not normally be physically deleted.
- Notifications should not bypass authorization.
- WhatsApp-originated task updates must follow the same authorization/business rules as web-originated updates.
- Passwords must be securely hashed and never returned in API responses.
- Search, filtering and pagination must be applied server-side within the user's authorized scope.

Standard response format

```
Success:
{
  "success": true,
  "message": "Task created successfully",
  "data": {}
}

Error:
{
  "success": false,
  "message": "You are not authorized to perform this action",
```

```
"errorCode": "FORBIDDEN"  
}
```

Standard HTTP status codes: 200, 201, 400, 401, 403, 404, 409, 422, 429, 500.

2. 20-Day Development Plan

Day	Focus	Main Deliverable
1	Backend Foundation + Auth	Login, logout, me, password APIs
2	Authentication Security	JWT/auth middleware, RBAC, validation, error handling
3	Users	User CRUD/update/status/task-view APIs
4	Departments	Department management APIs
5	Tasks	Create and list tasks
6	Task Details	Get/update task
7	Task Lifecycle	Cancel and reopen
8	Assignments	Assign task, list assignments, my assignments
9	Assignment Status	Status transition API
10	Assignment Operations	Acknowledge/complete/reassign
11	History + Audit	Task history retrieval and automatic history generation
12	Notifications	List/read/unread APIs
13	Dashboard	Role/scope-based dashboard
14	Reports	Task and performance reports
15	Subtasks	Subtask and progress APIs
16	Recurring Tasks	Recurring definition lifecycle APIs
17	Comments	Conditional comment APIs
18	Attachments	Conditional secure file APIs
19	WhatsApp	Webhook integration endpoint
20	Final Testing	Full Postman regression + security testing

The sequence follows the dependency order: authentication → users/departments → tasks → assignments/status → history/notifications → reporting → advanced features.

DAY 1 — Backend Foundation + Authentication

APIs

POST /api/auth/login

Body: {"email":"employee@example.com","password":"password"}

POST /api/auth/logout

Protected endpoint.

GET /api/auth/me

Returns the authenticated user.

POST /api/auth/change-password

Body: {"currentPassword":"oldPassword","newPassword":"newPassword"}

POST /api/auth/forgot-password

Public; exact email/OTP mechanism is to be finalized.

POST /api/auth/reset-password

Public with valid reset token/OTP.

Postman collection

- Auth → Login
- Auth → Me
- Auth → Logout
- Auth → Change Password
- Auth → Forgot Password
- Auth → Reset Password

DAY 2 — Authentication Security + Middleware

Build

- authMiddleware
- role/authorization middleware
- validation middleware
- centralized error middleware
- secure password hashing
- authentication/session/token handling

Authorization model

- SUPER_ADMIN → organization scope
- ADMIN → authorized department scope
- EMPLOYEE → own assignment scope

Postman tests

- No token → 401
- Invalid token → 401
- Unauthorized role/scope → 403
- Invalid input → 422/400

DAY 3 — Users APIs

APIs

GET /api/users

Supports page, limit, role, departmentId, status, search.

GET /api/users/:userId

Returns user details if authorized.

POST /api/users

Example: name, email, mobile, role, departmentId.

PATCH /api/users/:userId

Updates permitted fields.

PATCH /api/users/:userId/status

Body: {"status":"INACTIVE"}

GET /api/users/:userId/tasks

Returns tasks visible to the selected user.

Postman tests

- SUPER_ADMIN access
- ADMIN scope filtering
- Unauthorized department access
- Duplicate email
- Deactivate user and verify login is blocked

DAY 4 — Department APIs

APIs

GET /api/departments

Departments available to requester.

GET /api/departments/:departmentId

Department details.

POST /api/departments

Body: {"name":"IT","code":"IT","description":"Information Technology"}

PATCH /api/departments/:departmentId

Update department.

PATCH /api/departments/:departmentId/status

Activate/deactivate department.

PATCH /api/departments/:departmentId/head
Assign/change department head.

Postman tests

- Create department
- Update department
- Deactivate department
- Assign department head
- Verify Admin scope

DAY 5 — Core Task APIs

APIs

POST /api/tasks

Creates task + assignment(s) + history as one consistent business operation.

GET /api/tasks

Paginated, searchable, filtered and scope-controlled task list.

Create Task body

- {
-
- "
- t
- i
- t
- |
- e
- "
- :
- "
- P
- r
- e
- p
- a
- r
- e
-

- m
- o
- n
- t
- h
- l
- y
-
- r
- e
- p
- o
- r
- t
- "
- ,
-
- "
- d
- e
- s
- c
- r
- i
- p
- t
- i
- o
- n
- "
- :
- "
- P
- r
- e

- p
- a
- r
- e
-
- A
- u
- g
- u
- s
- t
-
- s
- a
- l
- e
- s
-
- r
- e
- p
- o
- r
- t
- "
- ,
-
- "
- p
- r
- i
- o
- r
- i
- t

- y
- "
- :
- "
- H
- I
- G
- H
- "
- ,
-
- "
- d
- u
- e
- D
- a
- t
- e
- "
- :
- "
- 2
- 0
- 2
- 6
- -
- 0
- 8
- -
- 2
- 0
- T
- 1
- 8

- :
- 0
- 0
- :
- 0
- 0
- Z
- "
- ,
-
- "
- d
- e
- p
- a
- r
- t
- m
- e
- n
- t
- l
- d
- "
- :
- "
- D
- E
- P
- A
- R
- T
- M
- E
- N

• T
• _
• l
• D
• "
• ,
•
• "
• a
• s
• s
• i
• g
• n
• m
• e
• n
• t
• s
• "
• :
• [
• {
• "
• a
• s
• s
• i
• g
• n
• e
• e
• l
• d
• "

- :
- "
- U
- S
- E
- R
- _
- I
- D
- "
- }
-]
-
- }

Postman tests

- Create as SUPER_ADMIN
- Create as ADMIN within department
- Try unauthorized assignee
- Invalid department
- Duplicate/invalid user
- Verify DB task + assignment + history

DAY 6 — Task Details + Update

APIs

GET /api/tasks/:taskId

Returns task, creator, department, assignments, parent/subtasks where permitted, status, progress, due date and priority.

PATCH /api/tasks/:taskId

Update title, description, priority, dueDate where permitted.

Postman tests

- Get valid task
- Get inaccessible task
- Update task
- Verify history for important changes

DAY 7 — Task Lifecycle

APIs

POST /api/tasks/:taskId/cancel

Authorized management users; validate state, update, history, notification.

POST /api/tasks/:taskId/reopen

Authorized management users; completed → reopened and create history.

Important

- Exact cancellation/reopen rules are not fully finalized in the requirements and must follow the approved business rules rather than invented rules.

DAY 8 — Task Assignment APIs

APIs

POST /api/tasks/:taskId/assignments

Body: {"assigneeIds":["USER_ID_1","USER_ID_2"]}

GET /api/tasks/:taskId/assignments

Returns assignments for a task.

GET /api/task-assignments/my

Authenticated employee's assignments.

GET /api/task-assignments/:assignmentId

Assignment details if requester has access.

Validation

- Task exists
- Requester authorized
- Users exist and are active
- Department/scope valid
- No duplicate assignment

DAY 9 — Assignment Status

API

PATCH /api/task-assignments/:assignmentId/status

Body: {"status":"IN_PROGRESS"}

Flow

- Authenticate → Find Assignment → Verify assignee/manager → Permission → Validate transition → Update → History → Notification

Example transitions

- ASSIGNED → ACKNOWLEDGED → IN_PROGRESS → COMPLETED

- Potential: IN_PROGRESS → BLOCKED → IN_PROGRESS

Important

- The exact status transition matrix is explicitly marked for finalization.

DAY 10 — Assignment Operations

APIs

POST /api/task-assignments/:assignmentId/acknowledge

Optional dedicated endpoint.

POST /api/task-assignments/:assignmentId/complete

Optional convenience endpoint; should reuse status business logic.

POST /api/task-assignments/:assignmentId/reassign

Body: {"newAssigneeId":"USER_ID"}

DELETE /api/task-assignments/:assignmentId

Not recommended for historical data; prefer cancel/revoke/reassign according to final rules.

DAY 11 — Task History + Audit

API

GET /api/tasks/:taskId/history

Chronological task history; supports page and limit.

Automatic history events

- Task created
- Task assigned
- Status changed
- Priority changed
- Reassigned
- Cancelled/reopened where applicable

Rule

- There should be no ordinary edit/delete API for task history. History is generated by business operations.

DAY 12 — Notifications

APIs

GET /api/notifications

Supports page, limit and unread=true.

GET /api/notifications/unread-count

Returns unread count.

PATCH /api/notifications/:notificationId/read

Marks one notification read after ownership check.

PATCH /api/notifications/read-all

Marks authenticated user's notifications as read.

Tests

- Only own notifications
- Unread filtering
- Read one
- Read all

DAY 13 — Dashboard

API

GET /api/reports/dashboard

Backend determines dashboard data from authenticated role/scope.

Expected scope

- SUPER_ADMIN → organization-wide users/tasks/departments summary
- ADMIN → department-scoped tasks/employees summary
- EMPLOYEE → personal tasks/upcoming summary

DAY 14 — Reports

APIs

GET /api/reports/tasks

Filters: fromDate, toDate, status, priority, departmentId, assigneeId.

GET /api/reports/employee-performance

SUPER_ADMIN organization-wide; ADMIN department-scoped.

GET /api/reports/department-performance

SUPER_ADMIN.

GET /api/reports/my-performance

Employee personal performance.

Rule

- Backend must enforce reporting scope.

DAY 15 — Subtasks

APIs

POST /api/tasks/:taskId/subtasks

Creates a subtask.

GET /api/tasks/:taskId/subtasks

Returns child tasks.

GET /api/tasks/:taskId/progress

Returns calculated progress.

Validation

- Parent exists
- Parent accessible
- Creator authorized
- Assignee valid
- No circular relationship

DAY 16 — Recurring Tasks

APIs

POST /api/recurring-tasks

Create recurring definition.

GET /api/recurring-tasks

List definitions.

GET /api/recurring-tasks/:id

Get definition.

PATCH /api/recurring-tasks/:id

Update definition.

POST /api/recurring-tasks/:id/pause

Pause future generation.

POST /api/recurring-tasks/:id/resume

Resume generation.

POST /api/recurring-tasks/:id/cancel

Cancel future occurrences.

Architecture

- Recurring Definition → Scheduler → Task Instance → Assignment → Notification

Important

- Exact recurring rules are still to be finalized.

DAY 17 — Comments (Conditional)

APIs

POST /api/tasks/:taskId/comments

Body: {"message":"Waiting for client approval."}

GET /api/tasks/:taskId/comments

Returns permitted comments.

PATCH /api/comments/:commentId

Edit if permitted.

DELETE /api/comments/:commentId

Delete only according to final business rules.

Important

- These are conditional because the specification says comments must be included only if finalized in the implementation scope.

DAY 18 — Attachments

APIs

POST /api/tasks/:taskId/attachments

Multipart upload.

GET /api/tasks/:taskId/attachments

List attachments.

GET /api/attachments/:attachmentId

Authorized download/view.

DELETE /api/attachments/:attachmentId

Authorized removal.

Validation

- File size
- MIME type
- Extension
- User permission
- Task access

Security

- Do not expose unrestricted public file URLs; file access must pass Taskify authorization.

DAY 19 — WhatsApp Webhook

API

POST /api/webhooks/whatsapp

Externally accessible provider webhook.

Flow

- Provider verification → signature validation → payload validation → sender identification → business authorization → identify task → validate status → update → history → notify

Important

- Exact provider and WhatsApp interaction model are not finalized in the requirements.

DAY 20 — Final Postman Testing + Security

Test every API

- Positive response
- Validation failure
- 401 authentication
- 403 authorization
- 404 not found
- 409 conflict
- 422 validation
- 429 rate limit
- 500 safe error handling

Cross-role tests

- SUPER_ADMIN organization access
- ADMIN department scope
- EMPLOYEE own assignment scope
- Inactive user cannot authenticate

Regression

- Create task → assignment → status change → history → notification → completion → dashboard/report update

3. Final Postman Collection Checklist

- 01 Auth: Login, Logout, Me, Change Password, Forgot Password, Reset Password
- 02 Users: List, Details, Create, Update, Status, User Tasks
- 03 Departments: List, Details, Create, Update, Status, Department Head
- 04 Tasks: Create, List, Details, Update, Cancel, Reopen, Subtasks, Progress
- 05 Assignments: Assign, List, My Assignments, Details, Status, Acknowledge*, Complete*, Reassign
- 06 History: Task History
- 07 Notifications: List, Unread Count, Read, Read All
- 08 Reports: Dashboard, Tasks, Employee Performance, Department Performance, My Performance
- 09 Recurring Tasks: Create, List, Details, Update, Pause, Resume, Cancel
- 10 Comments*: Create, List, Update, Delete
- 11 Attachments: Upload, List, Download/View, Delete
- 12 Webhooks: WhatsApp

* Conditional/optional endpoint according to the current API specification; implement only after the relevant business rule is finalized.

4. Important Business Rules to Verify During Testing

- Employee cannot delete an assigned task.
- Employee cannot assign tasks to another employee.
- Admin can assign tasks only within authorized department scope.
- Super Admin has organization-wide task-management authority.
- Employees can view only tasks they are authorized to view.
- Task history cannot be edited by ordinary users.
- Important historical records should not be physically deleted.
- Every important task-state change must generate a history event.
- WhatsApp-originated updates must follow the same authorization rules as web-originated updates.
- Frontend authorization is not the primary security boundary; backend authorization is mandatory.
- Inactive users cannot authenticate.
- Task assignment must validate recipient authorization.
- Task status transitions must be controlled.
- Notifications must not bypass authorization.

5. Recommended Backend Folder Structure

```
server/  
├─ src/
```

```
| ├── config/
| ├── controllers/
| ├── services/
| ├── models/
| ├── routes/
| ├── middlewares/
| ├── validations/
| ├── utils/
| ├── constants/
| └── jobs/
└── uploads/
└── .env
└── server.js
└── package.json
```

6. Definition of Done for Each API

- Route exists and is reachable.
- Authentication/authorization is applied where required.
- Input validation works.
- Controller delegates business logic to the service.
- Service enforces business rules and scope.
- Model/database operation works.
- Standard success response works.
- Expected errors return the correct HTTP status.
- Postman positive and negative tests pass.
- Important state changes create history/audit records.
- Notification side effects are triggered where required.

7. Important Scope Note

The source requirements explicitly leave several business decisions for finalization: exact task statuses and transitions, multi-employee task behavior, parent/subtask completion rules, recurring task rules, reassignment/cancellation/reopen rules, comments, attachments, WhatsApp interaction, email/SMS use cases, reports/KPIs, deactivation behavior and department management rules. This roadmap does not invent those missing rules.

Source basis: Taskify Business Requirements, Taskify Functional Requirements, Taskify High-Level Design, Taskify Database Design, and Taskify API Specification v1.0.